

Python 打包与 PyPI：概述（课堂讲义）

教学定位：本讲义对应 PyPA《Python Packaging User Guide》中的 [概述](#) 一节，是一张**「谁在什么环境运行 → 选哪种分发方式」的概念地图**，用来解释为何 Python 打包选项看起来很多。

与动手课的关系：从账号注册、构建 wheel / sdist、上传到 **TestPyPI/PyPI** 的**最小实操**，见同目录下的 `pypi-lab.md`；本文件可与官方文档对照阅读，**以官网为准**。

正文由 Sphinx/Furo 导出的单页 HTML 整理而来，**仅保留中文标签页**；原作者与版权归属 [PyPA](#)。

目录

- [概述](#)
- [考虑部署](#)
- [打包 Python 库和工具](#)
- [打包 Python 应用程序](#)
- [那么...](#)
- [总结](#)
- [课堂延伸：动手与资源](#)

概述

作为一种通用编程语言，Python 旨在多种方式使用。您可以用它构建网站、工业机器人或为朋友们制作游戏，等等，所有这些都可以使用相同的核心技术。

Python 的灵活性正是为什么每个 Python 项目的第一步必须是思考项目的受众和项目运行的环境。虽然在编写代码之前考虑打包可能看起来有些奇怪，但这个过程可以有效避免未来的麻烦。

本概述提供了一个通用的决策树，帮助您理清 Python 众多打包选项。继续阅读，选择最适合您下一个项目的技术。

考虑部署

软件包的存在是为了被安装（或 **部署**），因此在打包任何内容之前，您需要回答以下一些关于部署的问题：

- 谁是您的软件用户？您的软件是由从事软件开发的开发人员、数据中心的运维人员，还是是一些不太懂软件的用户群体安装？
- 您的软件是打算在服务器、桌面、移动客户端（手机、平板等）上运行，还是嵌入到专用设备中？
- 您的软件是单独安装，还是以大规模批量部署的方式安装？

打包与目标环境和部署体验密切相关。上述问题有很多不同的答案，每种情况组合都有其解决方案。根据这些信息，以下概述将帮助您选择最适合您项目的打包技术。

打包 Python 库和工具

您可能听说过 PyPI、历史上的 `setup.py` 以及 `wheel` 等概念。这些只是 Python 生态里「把代码交给别的开发者」相关工具的一部分；**当前官方推荐以 `pyproject.toml` 与标准构建前端（如 `build`）为主**，`setup.py` 多视为遗留路径。更多实践见 [打包和分发项目](#)。

以下的打包方法适用于在开发环境中由技术人员使用的库和工具。如果您正在寻找为非技术用户和/或生产环境打包 Python 的方法，请跳到 [打包 Python 应用程序](#)。

Python 模块

一个 Python 文件，只要它仅依赖于标准库，就可以重新分发和重用。您还需要确保它是为正确版本的 Python 编写的，并且只依赖于标准库。

这种方式非常适合在使用兼容 Python 版本的人之间共享简单的脚本和代码片段（例如通过电子邮件、StackOverflow 或 GitHub gists）。甚至有一些整个 Python 库也提供了这种选项，比如 [bottle.py](#) 和 [boltons](#)。

然而，对于由多个文件组成、需要额外库或需要特定版本 Python 的项目，这种模式就不适用了，因此下面列出了其他选项。

Python 源代码发行版

如果您的代码由多个 Python 文件组成，通常会以目录结构的形式进行组织。任何包含 Python 文件的目录都可以构成一个 [导入包](#)。

由于包由多个文件组成，它们的分发变得更为复杂。大多数协议仅支持一次传输一个文件（您上次点击链接并下载多个文件是什么时候？）。这增加了获取不完整传输的风险，并且更难确保目标位置的代码完整性。

只要您的代码仅包含纯 Python 代码，并且您知道您的部署环境支持您的 Python 版本，那么您可以使用 Python 的原生打包工具创建一个 [源代码分发包](#)，简称 *sdist*。

Python 的 *sdist* 是压缩档案文件（`.tar.gz` 文件），其中包含一个或多个包或模块。如果您的代码是纯 Python 的，并且只依赖其他 Python 包，可查阅官方 [源代码分发格式](#) 规范了解更多信息。

如果您的代码依赖任何非 Python 代码或非 Python 包（例如 `lxml` 中的 `libxml2`，或 `numpy` 中的 BLAS 库），您将需要使用下一节中介绍的格式，这对于纯 Python 库也有许多优点。

备注

Python 和 PyPI 支持多种分发方式，提供相同包的不同实现。例如，已不再维护但具有重要意义的 [PIL 分发包](#) 提供了 PIL 包，而 [Pillow](#) 是 PIL 的一个活跃维护的分支。

这一机制使得 Pillow 可以作为 PIL 的即插即用替代品，只需更改项目的 `install_requires` 或 `requirements.txt`。

Python 二进制发行版

Python 的实际能力在于它能够与软件生态系统集成，尤其是与用 C、C++、Fortran、Rust 和其他语言编写的库进行集成。

并不是所有开发者都有构建这些用编译语言编写的组件的工具或经验，因此 Python 创建了 **Wheel**（轮子）——一种旨在传输包含编译产物的库的包格式。事实上，Python 的包安装器 `pip` 总是优先使用 wheel 格式，因为安装速度更快，因此即使是纯 Python 包，使用 wheel 格式也能更好地工作。

二进制分发包在与源代码分发包一起提供时效果最好。即使您没有为每个操作系统上传您的代码的 Wheel 文件，通过上传 *sdist*，您仍然能让其他平台的用户自行构建您的代码。建议默认同时发布 *sdist* 和 wheel 文件，除非您正在为一个特定的用例创建构件，并且知道接收方只需要其中之一。

Python 和 PyPI 使得同时上传 Wheel 和 *sdist* 文件变得非常简单。只需按照 [打包Python项目](#) 中的教程操作。

 [Python 的工具和库打包能力概览](#)。

Python 推荐的内置库和工具打包技术。摘自 [The Packaging Gradient \(2017\)](#)。

打包 Python 应用程序

到目前为止，我们只讨论了 Python 的原生分发工具。根据我们的介绍，您可以推断出这些内置方法仅适用于具有 Python 环境并且懂得如何安装 Python 包的用户。

考虑到操作系统、配置和使用者的多样性，只有在面向开发者用户时，这种假设才是安全的。

Python 的原生打包方式主要是为了在开发者之间分发可重用的代码，即库。您可以利用 Python 的库打包技术，借助像 [setuptools entry_points](#) 这样的工具，将 **工具** 或开发者的基础应用程序构建在其上。

库是构建块，而不是完整的应用程序。对于应用程序的分发，存在着一系列全新的技术可供选择。

接下来的几个部分将根据目标环境的依赖关系来整理这些应用程序打包选项，帮助您为项目选择合适的解决方案。

取决于框架

某些类型的 Python 应用程序，如网站后端和其他网络服务，足够常见，因此它们有相应的框架来支持开发和打包。其他类型的应用程序，如动态网页前端和移动客户端，因其复杂性而使框架不仅仅是一个便利工具。

在所有这些情况下，从框架的打包和部署方式出发倒推是一个合理的做法。一些框架包含了一个部署系统，这个系统封装了本指南中提到的技术。在这种情况下，您将希望参考框架的打包指南，以便获得最简单和最可靠的生产体验。

如果您曾经好奇这些平台和框架是如何在幕后工作的，您可以随时阅读接下来的章节。

服务平台

如果您正在为“[平台即服务\(Platform-as-a-Service : PaaS\)](#)”开发应用程序，您需要遵循各自的打包指南。这类平台会处理打包和部署，只要您遵循它们的模式。大多数软件并不符合这些模板，因此才会有下面介绍的其他选项。

如果您正在开发将部署到您自己拥有的机器、用户的个人计算机或其他任何安排的应用程序，请继续阅读。

Web 浏览器和移动应用程序

Python 的持续进步正在将它引入新的领域。如今，您可以使用 Python 编写移动应用程序或 Web 应用程序的前端。虽然语言本身可能很熟悉，但打包和部署的实践却是全新的。

如果您计划在这些新领域发布应用程序，您将需要查看以下框架，并参考它们的打包指南：

- [Kivy](#)
- [Beeware](#)
- [Brython](#)
- [Flexx](#)

如果您不打算使用框架或平台，或者仅仅对上述框架使用的一些技术和方法感兴趣，继续阅读下面的内容。

取决于预安装的 Python

选择一台任意计算机，根据上下文，很有可能 Python 已经预先安装。多年来，Python 默认包含在大多数 Linux 和 Mac 操作系统中，因此您可以合理地依赖它已经存在于数据中心或开发者和数据科学家的个人机器上。

支持此模型的技术包括：

- [PEX](#)（Python 可执行文件）
- [zipapp](#)（不帮助管理依赖项，要求 Python 3.5+）
- [shiv](#)（要求 Python 3）

备注

在这里的所有方法中，依赖预安装 Python **最依赖目标环境**。当然，这也使得包的体积最小，可能只有几兆字节，甚至是几千字节。

一般来说，减少对目标系统的依赖会增加包的大小，因此这些解决方案大致按**输出体积递增**的顺序排列。

取决于单独的软件发行生态系统

长期以来，许多操作系统，包括 Mac 和 Windows，都缺乏内置的包管理功能。直到最近，这些操作系统才获得了所谓的“应用商店”，但即便如此，这些商店也主要集中于消费者应用程序，对于开发者的支持非常有限。

开发者长期寻求解决方案，在这场斗争中，他们推出了自己的包管理工具，如 [Homebrew](#)。对于 Python 开发者来说，最相关的替代方案是一个叫做 [Anaconda](#) 的包生态系统。Anaconda 以 Python 为基础，已经在学术、分析和其他数据导向的环境中越来越普遍，甚至逐步进入了 [面向服务器的环境](#)。

关于如何为 Anaconda 生态系统构建和发布的说明：

- [使用 conda 构建库和应用程序](#)
- [将原生 Python 包迁移到 Anaconda](#)

另一种类似的模式是安装一个替代的 Python 发行版，但不支持操作系统级别的任意包：

- [ActiveState ActivePython](#)
- [WinPython](#)

自带 Python 可执行文件

计算机的定义是能够执行程序。每个操作系统都原生支持一种或多种可以本地执行的程序格式。

有许多技术和工具将你的 Python 程序转换为这些格式，大多数方法涉及将 Python 解释器和其他依赖项嵌入到一个单独的可执行文件中。

这种方法被称为 *冻结(freezing)*，它提供了广泛的兼容性和无缝的用户体验，但通常需要多种技术，并且需要投入相当的精力。

以下是一些 Python 冻结工具：

- [pyInstaller](#) - 跨平台
- [cx_Freeze](#) - 跨平台
- [constructor](#) - 用于命令行安装程序
- [py2exe](#) - 仅限 Windows
- [py2app](#) - 仅限 Mac
- [osnap](#) - Windows 和 Mac
- [pysist](#) - 仅限 Windows

上述大多数工具都假设是单用户部署。对于多组件服务器应用程序，参见 [Chef Omnibus](#)。

自带用户空间

越来越多的操作系统——包括 Linux、Mac OS 和 Windows——可以通过使用一种相对较新的方式，称为 [操作系统级虚拟化](#) 或 [容器化\(containerization\)](#)，来运行作为轻量级镜像打包的应用程序。

这些技术大多与 Python 无关，因为它们打包的是整个操作系统文件系统，而不仅仅是 Python 或 Python 包。

这种技术在 Linux 服务器中应用最为广泛，因为它最初源于此，并且以下技术在这里效果最佳：

- [ApplImage](#)
- [Docker](#)
- [Flatpak](#)
- [Snapcraft](#)

自带内核

大多数操作系统支持某种形式的经典虚拟化，运行打包为包含完整操作系统的镜像的应用程序。这些虚拟机（VM）的运行是一种成熟的方法，在数据中心环境中广泛应用。

这些技术主要用于数据中心的大规模部署，尽管某些复杂应用程序可以从这种打包方式中受益。这些技术与 Python 无关，包括：

- [Vagrant](#)
- [VHD](#)、[AMI](#) 和 [其他格式](#)
- [OpenStack](#) - 一种 Python 编写的云管理系统，具有广泛的虚拟机支持

自带硬件

将软件以已安装在硬件上的方式进行交付，是最全面的方式。通过这种方式，用户只需要电力即可使用您的软件。

与上述主要面向技术人员的虚拟机不同，硬件设备已被广泛应用，从最先进的数据中心到最年轻的儿童都能看到其身影。

您可以将代码嵌入到 [Adafruit](#)、[MicroPython](#) 或其他更强大的运行 Python 的硬件中，然后将其交付到数据中心或用户家中。他们插电即用，您也可以高枕无忧。

 [用于打包 Python 应用程序的技术概述](#)。

用于打包 Python 应用程序的技术的简化范畴。

那么...

上面的部分只能总结这么多，您可能会对一些更明显的差距感到疑惑。

操作系统包

如在 [取决于单独的软件发行生态系统](#) 中提到的，某些操作系统拥有自己的包管理器。如果您非常确定目标操作系统，可以直接依赖像 [deb](#)（用于 Debian、Ubuntu 等）或 [RPM](#)（用于 Red Hat、Fedora 等）这样的格式，并使用该内置包管理器来处理安装，甚至部署。您甚至可以使用 [FPM](#) 从相同的源生成 deb 和 RPM 包。

在大多数部署流水线中，操作系统包管理器只是其中的一部分。

虚拟环境

[Virtualenvs](#) 一直是多代 Python 开发者不可或缺的工具，但随着更高层次的工具逐渐取代它们，虚拟环境的使用逐渐淡出了视野。在包装方面，虚拟环境作为一种原始工具，通常被包装在 [dh-virtualenv 工具](#) 和 [osnap](#) 中，二者都以自包含的方式包装了虚拟环境。

对于生产部署，切勿像在开发环境中那样依赖使用 `python -m pip install` 从互联网安装包到虚拟环境中。上面的概述提供了许多更好的解决方案。

安全

随着您深入了解这一梯度，更新包的组件变得越来越困难，因为所有内容都被更加紧密地绑定在一起。

例如，如果出现内核安全问题并且您部署的是容器，那么主机系统的内核可以更新，而无需为应用程序重新构建。如果您部署的是虚拟机镜像，则需要重新构建。关于这一动态是否使某种选项更安全，仍然是一个存在争议的老问题，回溯至尚未解决的 [静态链接与动态链接的对比](#)。

总结

Python 的打包系统有时给人一种颠簸的感觉。这种印象主要是因为 Python 的多功能性而产生的副作用。一旦你理解了不同打包方案之间的自然边界，你会发现，Python 程序员为了使用这种最平衡、最灵活的语言之一，而付出的代价，实际上是非常小的。